



COMP4601 (2026 T2)

Final Report

Topic: FPGA Acceleration of JPEG-Style Grayscale Image Compression

zID	Name
z5625451	Krishna Mittal
z5400462	Sakina Huzaifa
z5581147	Abhishek Poddar
z5619441	Shriya Malik

Executive summary

The submitted system compresses 512 x 512, 8-bit binary PGM images using an 8 x 8 JPEG-style pipeline: level shifting, a separable two-dimensional discrete cosine transform (DCT), quantisation, a project-specific zigzag permutation and zero-run run-length encoding (RLE). The result is stored in a custom little-endian GSC1 container. The archive contains one CPU compressor, one CPU decompressor, an XRT host program and one top-level HLS hardware kernel composed of four streaming dataflow stages.

Across 41 images (167,936 blocks), the CPU baseline required 1,418.062 ms, or 34.587 ms/image. The accelerated batch required 632.720 ms after XRT configuration and buffer allocation, giving 15.432 ms/image, 64.800 images/s, and a primary end-to-end speedup of 2.241x. The hardware kernel itself averaged 2.005 ms/image; compared with the closest CPU algorithmic stages (23.923 ms/image), this corresponds to an approximately 11.932x stage-level acceleration.

The main post-acceleration bottleneck is no longer the DCT or quantisation. From the displayed per-image rows, host-side GSC1 serialisation and file output average approximately 14.556 ms/image, about 85.6% of the non-overlapped estimated stage time. Double buffering reduces the summed stage time from 696.894 ms to an actual 632.720 ms batch, but the unaccelerated output path limits the overall gain.

Key measure	Observed result
Images/dimensions	41 images; 512 x 512 grayscale; 4,096 blocks/image
Raw/compressed size	10,747,904 bytes / 4,790,212 bytes
Compression ratio	2.244:1 (raw pixel payload divided by GSC1 file size)
CPU baseline	34.587 ms/image; 1,418.062 ms batch; 28.913 images/s
Accelerated batch	15.432 ms/image; 632.720 ms batch; 64.800 images/s
Hardware kernel	2.005 ms/image; approximately 130.745 megapixels/s
Primary speedup	2.241x actual pipelined end-to-end batch speedup

Central finding: The accelerator successfully removes the transform/coding bottleneck, but overall performance is now dominated by variable-length output construction and storage I/O.

1. Submitted codebase and compression format

All six source files in the final archive were reviewed. The host and kernel copies of `compression_header.hpp` are byte-identical and define the fixed image geometry, the 8 x 8 quantisation matrix, worst-case RLE storage and the top-level kernel interface.

File	Responsibility
<code>compression_3/Host/compression_header.hpp</code>	Constants, quantisation matrix and kernel declaration
<code>compression_3/Host/compression_host.cpp</code>	PGM loading, XRT setup, double buffering, profiling, GSC1 writing and CSV output
<code>compression_3/Kernel/compression_header.hpp</code>	Identical shared definitions used during HLS compilation
<code>compression_3/Kernel/compression_kernel.cpp</code>	Fixed-point DCT, four streaming stages, AXI interfaces and DATAFLOW top level
<code>cpu_only/compression_cpu.cpp</code>	Sequential software reference compressor and per-stage profiler
<code>cpu_only/decompression_cpu.cpp</code>	GSC1 parser, inverse RLE/zigzag/quantisation/DCT and PGM reconstruction
<code>cpu_only/compare_pixels.cpp</code>	Compares each original PGM image with its decompressed version and measures reconstruction quality using metrics such as MSE, MAE, PSNR, SNR, SSIM, and pixel-wise error statistics.

1.1 Algorithmic pipeline

PGM P5 load	→	Level shift (-128)	→	2D DCT CXC ^T	→	Quantise and round	→	Zigzag order	→	Zero-run RLE	→	GSC1 file
-------------	---	--------------------	---	-------------------------	---	--------------------	---	--------------	---	--------------	---	-----------

For each block, the level-shifted sample matrix X is transformed as $F = C X C^T$. Quantisation applies $q(u,v) = \text{round}(F(u,v) / M(u,v))$, where M is the stored 8 x 8 luminance matrix. The first coefficient is recorded as the DC value. The remaining 63 coefficients are represented only when non-zero, as {zero_run, value} pairs; trailing zeros require no explicit end-of-block symbol because each block stores its pair count.

1.2 Custom GSC1 binary layout

Field	Bytes	Meaning
Magic	4	ASCII GSC1
Image metadata	20	width, height, padded width, padded height and block count; five little-endian uint32 values
Quantisation matrix	128	64 little-endian uint16 values
Per-block fixed fields	3/block	int16 DC coefficient + uint8 number of AC pairs
Each non-zero AC pair	3/pair	uint8 preceding-zero count + int16 coefficient
Total size	variable	152 + 3 x blocks + 3 x total non-zero AC pairs

With 4,096 blocks, the minimum file is 12,440 bytes (no non-zero AC coefficients), and the worst-case reserved representation would be 786,584 bytes. The measured average is about 116,834 bytes/image. GSC1 is a self-consistent project format, not a standards-compliant JPEG bitstream: it has no colour conversion, chroma subsampling, differential DC coding, Huffman coding or JPEG marker structure. The zigzag array is retained exactly from the software implementation, so the compressor and decompressor remain compatible.

2. CPU baseline: implementation and profiling

The CPU compressor calculates the cosine transform matrix once, then visits all 64 x 64 block locations sequentially. For every block, it constructs a double-precision level-shifted array, performs two matrix-multiplication passes for the DCT, divides by the quantisation matrix, applies `std::lround` and int16 saturation, reorders the values, and appends an RLE object containing one dynamic vector of AC pairs. After all 4,096 blocks are accumulated, `saveCompressedimage` writes the GSC1 header and every block record. A monotonic `STD::chrono::steady_clock` is placed around image loading, each block stage, and file writing. Stage totals are accumulated across 167,936 blocks, divided by 41 for ms/image, and divided by total blocks for us/block. Other overhead is the residual between the complete per-image wall time and the explicitly timed stages; it therefore includes loop/control costs, repeated clock calls, container management, and operations outside the timed regions.

CPU stage	ms/image	us/block	Share of total
Image loading	0.362	-	1.046%
Level shifting	1.236	0.302	3.575%
2D DCT	7.711	1.883	22.295%
Quantisation	7.804	1.905	22.562%
Zigzag scan	3.536	0.863	10.222%
RLE encoding	3.636	0.888	10.512%
File writing	8.343	-	24.122%
Other overhead	1.960	-	5.665%
Total	34.587	-	100%

The DCT and quantisation together consume 15.515 ms/image (44.857%); adding level shift, zigzag and RLE gives 23.923 ms/image for the stages moved into the hardware kernel. File writing is the largest single CPU stage at 8.343 ms/image. The reported 34.587 ms/image and 1,418.062 ms batch are mathematically consistent: $1,418.062 / 41 = 34.587$ ms/image after rounding.

2.1 Meaning of the CPU terminal lines

Displayed line	Interpretation
Compression complete	All input directory entries have been processed without an exception.
imgs compressed: 41	Number of files passed through compressing; the current folder contains 41 PGM images.
Input format: PGM P5	The intended input is binary, 8-bit grayscale PGM.
Processed dimensions / Blocks per img	The benchmark reports 512 x 512 pixels and 4,096 non-overlapping 8 x 8 blocks.

Displayed line	Interpretation
Total raw img size: 10747904 bytes	41 x 512 x 512 pixel bytes; PGM header bytes are deliberately excluded.
Total compressed size: 4790212 bytes	Sum of the on-disk GSC1 file sizes.
Overall compression ratio: 2.244:1	10,747,904 / 4,790,212; larger is better.
Average profiling results	Stage totals divided by 41 images; percentages use the full 1,418.062 ms.
Total blocks processed: 167936	41 x 4,096; denominator for each us/block value.
Batch processing time: 1418.062 ms	Sum of each image-level compressing elapsed time; setup and directory scanning are outside this total.

3. Accelerated host architecture and overlap strategy

The XRT host accepts three arguments: the compiled hardware binary, one PGM file or a directory, and the output directory. It opens XRT device 0, loads the binary, constructs `compression_stream_kernel`, and allocates two complete `BufferSet` objects. Each buffer is placed in the memory group connected to the corresponding kernel argument by `compression.group_id(argument_index)`. The top-level accelerated design therefore has one named hardware kernel; the four functions inside it are concurrent dataflow stages, not four separately launched kernels.

Mapped buffer	Size	Purpose
Input pixels	262,144 bytes	512 x 512 uint8
DC values	8,192 bytes	4,096 int16
Pair counts	4,096 bytes	4,096 uint8
Zero runs	258,048 bytes	4,096 x 63 uint8
Pair values	516,096 bytes	4,096 x 63 int16
One BufferSet total	1,048,576 bytes	256 KiB input + 768 KiB output
Two slots	2,097,152 bytes	Double-buffered host/device working storage

3.1 Double-buffered processing sequence

Phase	Current slot / CPU activity	Other slot / device activity
Initialisation	Load image 0; copy to slot 0; H2D sync; launch kernel 0	Idle
While kernel i runs	Kernel i executes in current slot	Load/copy/H2D for image i+1 in the other slot
After kernel i completes	Launch kernel i+1, then D2H and serialise/write image i	Kernel i+1 executes
Final drain	D2H and write last image	No next image to prepare

`prepareImage` measures three host-side stages: PGM file loading, copying the pixel vector into the mapped input BO, and `XCL_BO_SYNC_BO_TO_DEVICE`. `launchKernel` records a timestamp before invoking the XRT kernel and waits asynchronously in a `std::future`. This lets the main thread prepare the next image while the current run is active. Once the current run completes, the next run is launched before the previous result is copied back and written, so device execution overlaps the preceding image's D2H and file output. The code never has two kernel runs active at once due to FPGA hardware resource limitations; inter-image gain comes from host/device overlap, while intra-image concurrency comes from HLS DATAFLOW.

3.2 What is included and excluded from accelerated timing

Timing scope	Definition
Included in actual pipelined batch	First image load through last GSC1 write, including console/CSV reporting performed inside the loop.
Excluded from actual pipelined batch	Directory enumeration, XRT device opening, hardware binary loading, kernel construction, BO allocation/mapping and CSV-file creation.
Kernel time	Host-observed time from immediately before <code>xrt::kernel</code> invocation until <code>run.wait</code> completes; includes launch/wait overhead around one execution.

Timing scope	Definition
D2H behaviour	All four output BOs are synchronised, so 768 KiB is transferred per image even when the final compact GSC1 file is much smaller.
File-write timer	Includes counting valid pairs, allocating and filling a contiguous byte buffer, and the final filesystem write; it is not pure storage-device time.

4. Hardware kernel: stages, interfaces and HLS techniques

compression_stream_kernel exposes five independent AXI4 master bundles (gmem0-gmem4) plus an AXI4-Lite control interface. The independent bundles provide separate AXI master ports for the input, DC values, pair counts, zero runs and pair values. Burst lengths and up to 16 outstanding transactions are requested. Five HLS streams connect the internal stages, and #pragma HLS DATAFLOW schedules those stages to operate concurrently on different portions of the image.

Internal stage	Data movement	Detailed operation
1. load_and_shift_stage	DDR input -> 8-row BRAM strip -> block-order pixel stream	Loads each 8-row band once, subtracts 128 into signed 9-bit samples, then emits 64 pixels per block. Row dimension is completely partitioned; inner loops target II=1.
2. dct_stage	Pixel stream -> 8 x 8 local block -> DCT stream	Reads 64 samples, executes a separable two-pass DCT, then emits coefficients using the shared project zigzag table. Input, output, temporary arrays and DCT matrix are completely partitioned.
3. quantise_and_rle_stage	DCT stream -> DC/count/pair streams	Replaces division with multiplication by precomputed reciprocal quantisers, performs signed half-away-from-zero rounding with int16 saturation, and buffers valid AC pairs in dual-port LUTRAM.
4. write_results_stage	Streams -> four output DDR arrays	Writes one DC and one count per block, followed by only count valid pairs into the block's reserved 63-slot region.

4.1 Fixed-point design

Type	Declaration	Role
pixel_t	ap_int<9>	-128 to +127 level-shifted pixels
dct_coefficient_t	ap_fixed<18,2>	Precomputed cosine coefficients
first_stage_t	ap_fixed<25,11>	First one-dimensional DCT products/sums
second_stage_accumulator_t	ap_fixed<44,14>	Wide second-pass accumulation
dct_value_t	ap_fixed<32,13>	Stored DCT coefficient stream
reciprocal_t	ap_ufixed<24,1>	1 / quantisation value
quantisation_product_t	ap_fixed<56,14>	High-precision value x reciprocal before explicit integer rounding

All fixed-point types use AP_RND_CONV and AP_SAT for conversions, while quantise_and_round explicitly implements the same half-away-from-zero rule as std::lround and clamps to int16. This avoids floating-point hardware and runtime division while maintaining a controlled numerical path. Exact bit-for-bit equality with the double-precision CPU output is not demonstrated by the terminal captures and should be verified separately.

4.2 Main optimisation techniques

- **Precomputation:** DCT coefficients and reciprocal quantisation values are constants; no cosine or division is evaluated in hardware.
- **Parallel dot products:** Each eight-term dot product is manually expanded into eight multiplications and a balanced adder tree, then inlined.
- **Complete array partitioning:** Small 8 x 8 arrays become independently accessible registers/banks, enabling the dot-product parallelism.
- **II=1 loop pipelining:** Pixel loading, block reads, coefficient output, quantisation and valid-pair writes are requested at one iteration per cycle.
- **Streaming dataflow:** BRAM tiling, DCT, quantisation/RLE and DDR output operate as a producer-consumer pipeline with explicit FIFO depths.
- **Memory concurrency:** Separate AXI bundles, restrict-qualified pointers, burst settings and outstanding requests improve external-memory scheduling.

5. Accelerated terminal and CSV output: line-by-line interpretation

```
./compression_host compression_3.bin images output_3
Opening XRT device 0
Loading "compression_3.bin"
all_128.pgm | Total: 5.300 ms | Kernel: 2.018 ms | Write: 2.457 ms
...
Total kernel time for 41 images: 82.202 ms
Average kernel time per image: 2.005 ms
Total estimated stage time for 41 images: 696.894 ms
Average estimated stage time per image: 16.997 ms
Actual pipelined batch time: 632.720 ms
```

Output item	Meaning
Total	estimated_end_to_end_avg_ms = PGM load + mapped host copy + H2D + kernel + D2H + GSC1 serialisation/write. It is a sum of measured stages and does not subtract overlap.
Kernel	One launch-to-completion measurement for compression_stream_kernel. The CSV labels avg/min/max, but timed_runs is 1, so all three are identical for each image.
Write	Time inside saveGsc1: validate/sum counts, allocate compact buffer, serialise the header and valid block records, and write the file.
Total kernel time	Sum of the FPGA kernel execution times for all 41 images.
Average kernel time	Average FPGA kernel execution time required to process one image. $82.202 / 41 = 2.005$ ms/image.
Total estimated stage time	Sum of all individual per-image stage times as if no overlap occurred between images. Calculated by adding each image's PGM load + host copy + H2D + kernel + D2H + file write times, then summing these per-image totals across all 41 images.
Average estimated stage time	Average non-overlapped end-to-end processing time for one image. $696.894 / 41 = 16.997$ ms/image.
Actual pipelined batch time	Real wall-clock time to process all 41 images with the double-buffered host/device overlap enabled. Profiled using one wall-clock timer started before the double-buffered processing loop and stopped after all 41 images were completely processed.

5.1 stream_profile.csv fields

CSV column group	Definition
raw_bytes / compressed_bytes / compression_ratio	Pixel payload, GSC1 file size, and raw_bytes divided by compressed_bytes.
pgm_load_ms / host_copy_ms	Filesystem PGM load and CPU copy into the mapped BO.
h2d_ms / d2h_ms	Explicit BO synchronisation to and from the device.
kernel_avg/min/max_ms	One observed kernel time copied into all three columns because timed_runs = 1.
pipeline_with_transfer_avg_ms	H2D + kernel + D2H; despite the name, it is one run.
estimated_end_to_end_avg_ms	Load + host copy + transfer pipeline + file write; the terminal Total field.
kernel_mpix_per_s	0.262144 megapixels divided by kernel seconds.

6. Performance comparison

Measurement	41-image total	Per image	Throughput	Scope
CPU complete compression	1,418.062 ms	34.587 ms	28.913 images/s	Sequential baseline
FPGA estimated stage sum	696.894 ms	16.997 ms	58.834 images/s equivalent	No overlap deducted
FPGA actual pipelined batch	632.720 ms	15.432 ms	64.800 images/s	Primary end-to-end result
FPGA hardware kernel	82.202 ms	2.005 ms	130.745 MP/s	Kernel-only scope

6.1 Speedup calculations and fair interpretation

Metric	Calculation	Result	Interpretation
Actual end-to-end batch speedup	1,418.062 / 632.720	2.241x	Best primary comparison because both include image load, compression, and output file creation within their measured loops.
Estimated non-overlapped speedup	34.587 / 16.997	2.035x	Compares CPU average with the sum of accelerated stage timers before overlap.
Closest algorithmic-stage speedup	$(1.236 + 7.711 + 7.804 + 3.536 + 3.636) / 2.005$	11.932x	Shows the effect of moving level shift, DCT, quantisation, zigzag and RLE into hardware; scopes are close but not perfectly identical because the kernel also performs DDR input/output.
Pipeline overlap benefit	696.894 - 632.720	64.174 ms (9.209%)	Time hidden by double buffering, net of reporting overhead included in the actual loop.

7. Data-dependent behaviour and the new bottleneck

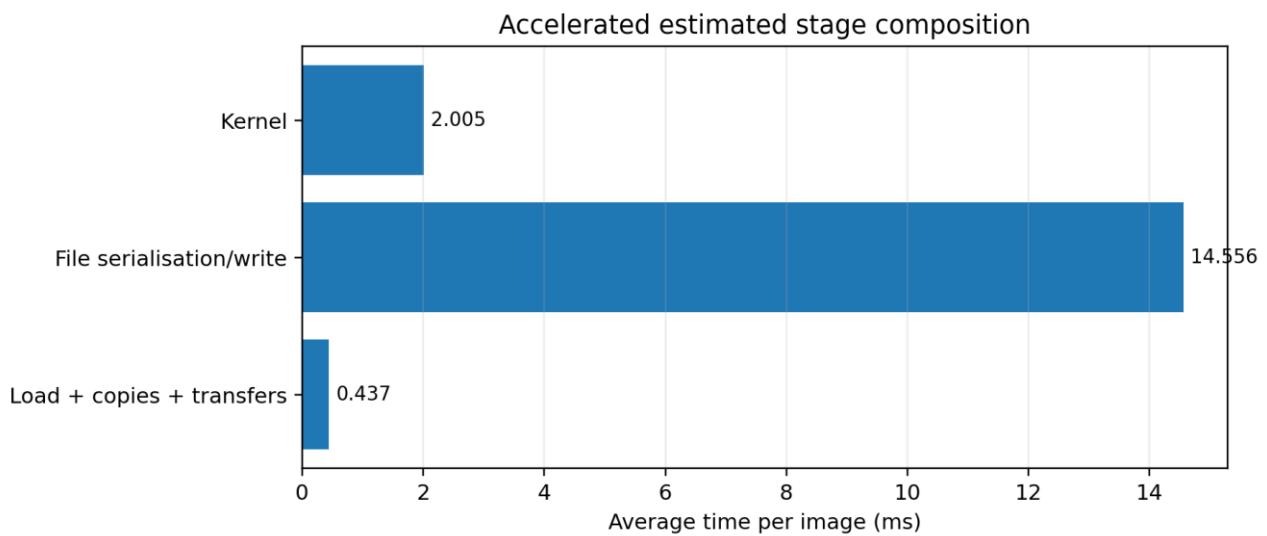


Figure 1. Average accelerated Total components derived from the 41 displayed rows. Rounding may change the final few microseconds.

Estimated component	Average	Share	Interpretation
Kernel	2.005 ms	11.8%	Mostly fixed transform workload; rises when many non-zero AC pairs must be emitted.
GSC1 serialisation/write	14.556 ms	85.6%	Strongly data- and filesystem-dependent; now dominates estimated end-to-end time.
Load, copies and transfers	0.437 ms	2.6%	Residual Total - Kernel - Write; combines PGM load, mapped copy, H2D and D2H.

Representative image	Total ms	Kernel ms	Write ms	Why it behaves this way
all_white.pgm	4.494	1.894	2.217	Very sparse quantised AC output; lowest displayed Total.
checkerboard_4px.pgm	22.052	2.027	19.631	Fine high-frequency pattern generates many retained coefficients.
t080.pgm	37.349	2.282	34.658	Complex natural/test image; output handling dominates.
fixed_seed_noise.pgm	69.663	2.951	66.319	Random noise produces the densest coefficient stream and the maximum Total.
vertical_gradient.pgm	5.995	1.898	3.708	Smooth signal concentrates energy into few low-frequency terms.

Kernel time varies from about 1.883 to 2.951 ms in the visible rows, whereas Total varies from 4.494 to 69.663 ms. The DCT always performs the same block count, but the quantise/RLE and output stages execute variable-length loops for valid AC pairs. The much larger Total variation shows that compact-file construction and storage are the dominant content-sensitive operations.

7.1 Highest-value next optimisations

- **Produce a compact byte stream in hardware:** write valid GSC1 records into one packed output buffer and return the exact byte count, avoiding four worst-case BOs and most host serialisation.
- **Decouple file I/O:** hand completed buffers to a dedicated writer thread or use asynchronous I/O so the CPU can prepare future images while storage is busy.
- **Queue multiple runs or CUs:** the current host waits for run i before launching run $i+1$. Multiple in-flight kernel runs could exploit additional compute units if the final bitstream and memory bandwidth support them.
- **Reduce transfer volume:** the host currently synchronises 768 KiB of output for every image, even when the compact file is only tens of kilobytes.

8. Decompression path and correctness

8.1 CPU decompressor

The decompressor is a complete inverse path for GSC1. It validates the four-byte magic, dimensions, padded dimensions, block count, non-zero quantisation entries, pair count and run bounds. For each block it reconstructs the 64-entry coefficient sequence, applies the same zigzag permutation in reverse, multiplies by the stored quantisation matrix, computes $X = C^T F C$, adds 128, rounds, clamps to $[0, 255]$, and places the 8×8 result into the reconstructed image. It writes a binary PGM P5 file and reports one total decompression time per file; it does not provide per-stage decompression profiling.

Inverse stage	Operation
GSC1 header parse	Validate metadata and read the stored matrix
RLE decode	Reinsert zero runs and non-zero AC coefficients; DC is coefficient 0
Inverse zigzag	Map the 64 values back to the 8×8 coefficient matrix
Dequantisation	Multiply each coefficient by its quantisation value
Inverse DCT	$C^T F C$ using double precision
Pixel reconstruction	Add 128, round, clamp and save PGM P5

8.2 Pixel-level verification with compare_pixels

Section 8.1 establishes that the decompressor inverts the GSC1 container correctly in structural terms. It does not establish how close the reconstructed pixels are to the originals, which is the question that ultimately matters for a lossy codec. We therefore include a second CPU utility, `compare_pixels`, which performs an objective pixel-level comparison between the original PGM images and the reconstructions produced by the decompressor.

The tool takes two directory arguments, `compare_pixels <original_image_dir> <decompressed_image_dir>`. It enumerates every .pgm file in the original directory and locates the corresponding reconstruction by appending `_stream` to the file stem, so that `images/t005.pgm` is paired with the reconstruction `t005_stream.pgm`. This matches the naming convention used throughout the pipeline: the compression host writes `NAME_stream.gsc`, and the decompressor preserves that stem when writing `NAME_stream.pgm`. Pairs whose partner file is missing, or whose dimensions disagree, are reported individually and flagged in the status column rather than aborting the run.

For each pair the utility prints a summary line to the terminal and appends a row to `image_comparison_metrics.csv` in the working directory. A final AGGREGATE row pools every pixel in the set, so the summary statistics are weighted by pixel count rather than being an unweighted mean of per-image values. The columns are as follows.

Metric	Definition and interpretation
mse	Mean squared error over all pixels. The base quantity from which rmse, psnr and snr are derived. Zero indicates bit-exact reconstruction.
rmse	Square root of mse, expressed in grey levels, so it is directly comparable with the 0-255 pixel range.
mae	Mean absolute error. Less sensitive than rmse to a small number of large deviations, so a large gap between the two indicates outliers rather than uniform error.
mean_error	Signed mean of (reconstructed - original). A value near zero indicates unbiased quantisation; a persistent non-zero value indicates systematic brightness drift.
max_absolute_error	Largest single-pixel deviation in the image. Identifies worst-case local damage that averaged metrics conceal.

psnr_db	Peak signal-to-noise ratio, $10 \log_{10}(255^2 / \text{mse})$. The conventional headline figure for image codecs. Reported as inf when mse is zero.
snr_db	Signal-to-noise ratio referenced to the actual signal power of the image rather than the fixed 255 peak. Substantially lower than psnr for dark or low-variance images.
mean_ssim_8x8	Structural similarity averaged over 8 x 8 windows, aligned with the transform block grid. Ranges from 0 to 1 and correlates better with perceived quality than the error-based metrics.
exact_pixels, exact_percent	Count and percentage of pixels reconstructed with zero error.
within_1_percent, within_2_percent	Percentage of pixels within 1% and 2% of the full 255 range, that is within approximately 2.55 and 5.1 grey levels.
metric_time_ms	Wall-clock time to compute all metrics for that image. Diagnostic only; it forms no part of the compression or decompression cost.
status, error	Ok on success, otherwise a short description of the failure for that pair.

8.3 Measured reconstruction quality on the 41-image set

The figures below were produced by decompressing the 41 GSC1 streams emitted by the accelerated compression_3 host run and comparing the reconstructions against the original images. Every pair completed with status ok; there were no missing files, dimension mismatches, or parse failures.

Aggregate metric	Value	Reading
Pixels compared	10,747,904	41 images at 512 x 512.
psnr_db	31.449	The low thirties is the conventional band for a DCT codec operating at this compression ratio.
mean_ssim_8x8	0.9371	Structure is well preserved; the losses are predominantly fine detail rather than structural.
rmse	6.825	A typical deviation of about 6.8 grey levels out of 255, roughly 2.7% of the range.
mae	3.990	Well below the rmse, indicating a light-tailed error distribution rather than widespread large deviations.
mean_error	-0.058	Effectively unbiased across 10.7 million pixels; the quantiser contributes symmetric rounding noise, not brightness drift.
max_absolute_error	87	Worst single pixel in the entire set, occurring in fixed_seed_noise.
exact_percent	21.556	Roughly one pixel in five survives the round trip completely unchanged.
within_2_percent	57.467	The majority of the remaining pixels deviate only slightly.

The synthetic patterns are the most informative individual rows, because their expected behavior can be predicted analytically and any deviation would indicate a defect. all_black and all_128 reconstruct bit-exactly, with zero mse, unity ssim and infinite psnr. A uniform field is pure DC, so every AC coefficient is zero and quantisation has nothing to discard; exact reconstruction is the correct outcome and confirms that the DC path, the dequantisation step and the inverse DCT are mutually consistent with the forward path.

all_white behaves differently in a way that is worth stating explicitly, since a reader who sees 0% exact pixels on the simplest possible image will otherwise suspect a fault. Every pixel is reconstructed exactly one grey level low, giving mse of exactly 1.0, a maximum absolute error of 1 and a mean_error of exactly -1.0. This is a deterministic rounding boundary rather than damage: after level shifting, a uniform 255 field produces a DC coefficient that falls just below a quantisation threshold, and the final rounding and clamping stage therefore returns 254. The perfect uniformity of the offset, and the fact that 100% of pixels remain within 1% of the range, distinguish this from an arithmetic error.

The checkerboard family isolates the frequency response of the 8 x 8 transform. The plain, 16-pixel and 64-pixel checkerboards all reconstruct with mse 0.5 and psnr 51.141 dB, because their edges align with the block grid and each block is again almost flat. The 4-pixel checkerboard is the stress case at 29.198 dB with a maximum error of 24: a period of four pixels sits close to the limit of an 8 x 8 DCT and forces signal energy into the highest-frequency coefficients, which is precisely where the quantisation matrix is coarsest. Its compressed stream was also the largest of the synthetic set at 159,896 bytes, so the codec pays for content at that frequency in both rate and distortion.

fixed_seed_noise is the worst case overall at 22.775 dB, with only 2.507% of pixels exact and the largest single-pixel error in the set. White noise carries no spatial correlation for the DCT to exploit, so the transform cannot compact its energy and the quantiser discards information that is not redundant. This image was also the only one whose compressed stream, at 555,986 bytes, exceeded the size of the raw 512 x 512 image. That is the expected pathological behavior of a transform codec on incompressible input rather than an implementation fault.

Impulse illustrates why psnr and snr must be read together. It records a very high psnr of 63.320 dB and 99.988% exact pixels, yet a snr of only 9.134 dB and a maximum error of 44. The image is almost entirely black with a small number of isolated bright pixels, so its total signal power is very low; a handful of errors concentrated at the impulses barely affects the fixed-peak psnr but dominates the signal-referenced snr. Both figures are correct, and their divergence is a property of the content rather than of the codec. Both gradients, by contrast, reconstruct to 47.168 dB with a maximum error of 2, since the lowest few DCT basis functions represent a linear ramp well. Across the thirty natural test images the results are consistent and unremarkable in the desirable sense. psnr spans 26.211 dB (t080) to 37.576 dB (t012) and ssim spans 0.8458 (t135) to 0.9728 (t096), with maximum per-pixel errors between 35 and 73. The spread tracks image content: images with large smooth regions sit at the top of the range, while those carrying dense high-frequency texture sit at the bottom. No natural image failed, exhibited anomalous bias, or fell outside the band predicted by the synthetic stress cases.

These measurements supply the objective quality evidence that timing results alone cannot. The speedup figures in Section 6 are only meaningful if the accelerated pipeline preserves reconstruction quality, and the aggregate 31.449 dB psnr and 0.9371 ssim, together with the analytically predictable behavior of every synthetic pattern, establish that the fixed-point HLS kernel produces output consistent with a correctly functioning JPEG-style codec at this compression ratio. One stronger claim remains available and is a natural extension of this work: running `compare_pixels` over reconstructions of the CPU compressor output and differencing the two metric sets would isolate any fidelity cost attributable specifically to the fixed-point kernel, as distinct from the lossy algorithm itself.

9. Conclusions and recommended final claims

What worked

- **Correct stage partitioning:** the most expensive CPU stages are moved into one streaming kernel, while file parsing and persistent output remain under host control.
- **Effective HLS architecture:** fixed-point arithmetic, parallel dot products, complete partitioning, II=1 pipelines, BRAM/LUTRAM buffering, separate AXI ports and DATAFLOW reduce the algorithmic path from 23.923 to about 2.005 ms/image.
- **Successful host overlap:** two BO sets hide 64.174 ms, or 9.209%, relative to the non-overlapped sum of accelerated stage timers.
- **Meaningful end-to-end gain:** the observed batch improves from 1,418.062 to 632.720 ms, a defensible 2.241x speedup and 64.800 images/s.
- **Compact, reversible project format:** the GSC1 file carries all metadata and the quantisation matrix needed by the supplied decompressor, achieving an overall 2.244:1 ratio on the 41-image set.

What did not scale and why

- **Host output path:** serialisation and filesystem writing average approximately 14.556 ms/image and dominate the accelerated Total. Once compute is fast, Amdahl's law makes further DCT-only optimisation low value.
- **Worst-case output buffers:** the kernel/host exchange always reserves and synchronises 63 pair slots per block, producing a fixed 768 KiB D2H transfer even for very sparse images.
- **Limited benchmark statistics:** one run per image and missing build/clock details prevent a rigorous variance or reproducibility analysis.
- **JPEG compatibility and quality evidence:** the project is JPEG-style rather than JPEG-compliant, and the final archive does not print objective reconstruction metrics.

Primary performance statement: On 41 512 x 512 grayscale images, the final KV260 implementation reduced measured batch compression time from 1,418.062 ms on the ARM CPU baseline to 632.720 ms with the double-buffered FPGA path, giving a 2.241x end-to-end speedup and 64.800 images/s. The hardware kernel averaged 2.005 ms/image, while host-side compact-file construction and writing became the dominant bottleneck.

Final assessment

The final design demonstrates a clear and technically sound acceleration result. Its strongest contribution is the transformation of a sequential per-block software pipeline into a fixed-point, streaming HLS dataflow kernel, combined with host-level double buffering. The 2.241x measured end-to-end speedup is the correct headline result. The much larger stage-level gain explains that the hardware itself is effective; the gap between the two numbers identifies the next engineering target unambiguously: compact output generation, transfer and storage rather than additional DCT arithmetic.